CUDA Project

Behzad Iqbal

May 2024

1 Introduction

As someone who enjoys making video slideshows from the photos I take during my travels, I wanted to make the process faster and smoother. I love sharing these memories on a big screen with my friends and family, making the experience more fun and visually appealing. I noticed that as I included more photos in my slideshows, it took longer to create them. To solve this problem, I decided to use GPU to speed up how images transition and how the videos are put together. This way, I can create videos more quickly and make it easier to share my travel memories with my loved ones. By using GPUs functionality to perform smooth transitions between a series of JPEG images and compiling them together I managed to achieve performance improvements over CPU-based approaches and thereby reduced the time to create video slideshows.

2 Implementation of code

In this project I used many different libraries to facilitate my process. Firstly, I used the libjpeg-turbo library. This library facilitates the process of decoding JPEG images, namely it allows us to extract pixel data and image data from the JPEG format images. Besides this library I used FFmpeg, a library that can be used for video creation and manipulation. Now moving to the functions:

- loadJPEGImage: This function is responsible for reading and decoding a JPEG file using the libjpeg-turbo library. It fetches the image dimensions and pixel data in RGB format and prepares the images for further processing.
- loadJPEGIMagesFromTextFile: This function reads a text file containing the different image filenames and loads the corresponding image files and fills the vectors with the image data, filenames, widths and heights. This function makes it easy to import multiple images from a specific folder.
- allocateDeviceMemoryForImages: This function allocates GPU memory for the images and copies the image data from the host to the devices. Furthermore this function optimizes the memory management for efficient and effective GPU processing.

- writeTransitionedJPEGImage: This function is responsible for writing the transitioned image data to a new JPEG image file using the libjpeg library for compressing and outputting the file. Additionally it also ensures that the processed images are saved in a format that is suitable for storage and sharing.
- createVideoFromImages: This function then generates a video file from a sequence of transitioned images using the library FFmpeg. On the other hand, this function creates a temporary text file, defines the parameters such as the frame rate and code and eventually executes an FFmpeg command to parse the images into a video file.

Besides the ordinary functions we have the kernel functions, which are responsible for the GPU tasks. First we have the functionality of applying the blackout transition effect to an input image. This involves the concept of darkening the entire image by scaling down the brightness of each pixel using a transition parameter. To implement this concept we have our first kernel function called 'blackoutTransitionKernel'. Firstly this kernel is launched with multiple threads containing organized block, each thread then corresponds to a pixel in the input image. After this each thread loads the pixel data from the input image into shared memory. The shared memory is then used for efficient data access and to minimize the memory latency. Then each thread modifies the value of the pixel by scaling down its brightness based on the transition variable. The lower the transition value gets, the darker the resulting pixel gets. Secondly we have the functionality of transitioning between two images, this involves the concept of blending the pixels from both images together using the transition factor ('transition'). The goal hereby is to create a smooth visual effect where one image over time gradually transitions into another image. To achieve this, we have our first kernel function called 'transitionBetweenImagesKernel'. This kernel launches multiple threads containing organized block, each thread then corresponds to a pixel in the output image. Then each thread loads the pixel data from two input images into shared memory. Next step is to utilize the loaded pixel values from both images and each thread will calculate the blended pixel value for the output image. The blending is again based on the transition factor, which is responsible for the contribution of each image's pixel to the final output of the image. The final pixel value gets computed by the concept of linearly interpolation. After the computation has been done, the output image now contains the transitioned pixels which then results into a smooth blend between the images. And finally all the functions get implemented in the main function to achieve the desired outcome of the project.

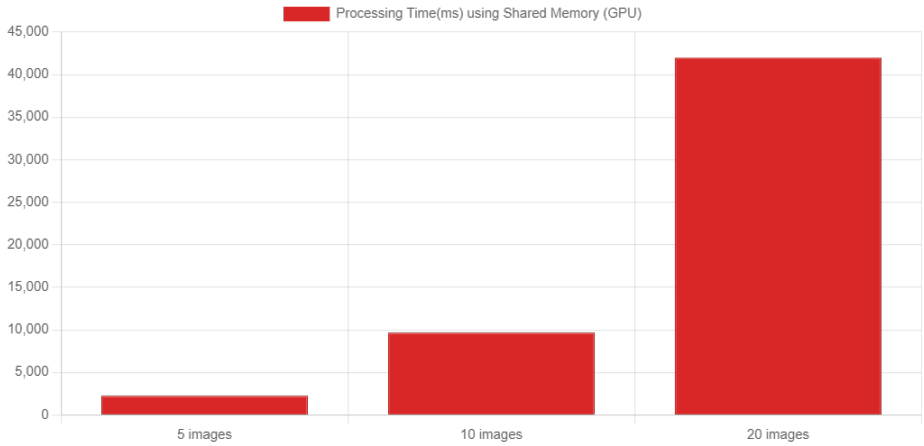


Figure 1: Processing time using Shared Memory (GPU)

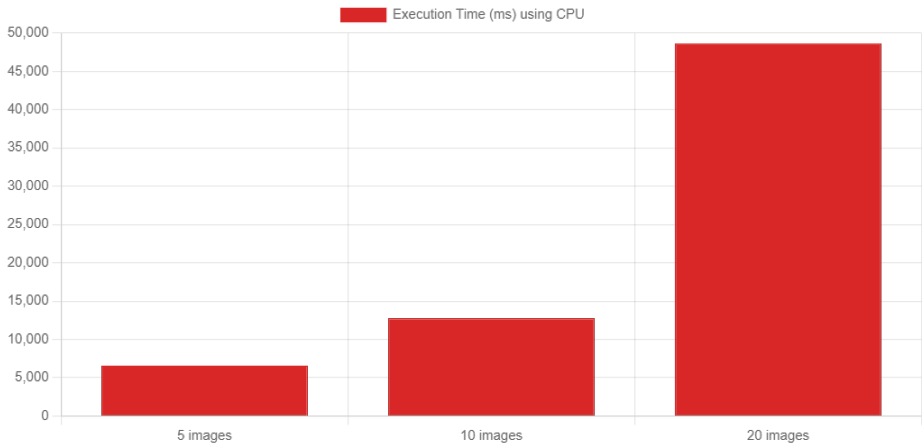


Figure 2: Processing time using CPU

3 Performance Comparison: GPU vs. CPU Image Processing

3.1 GPU Execution Times

The application of image transitions using the GPU gives us the following execution times:

- 5 Images: 2.24 seconds
- 10 Images: 9.7 seconds
- 20 Images: 42 seconds

3.2 CPU Execution Times

On the other hand, when applying the same image transitions using the CPU, the execution times are as follows:

- 5 Images: 6.6 seconds
- 10 Images: 12.8 seconds
- 20 Images: 49 seconds

A comparison of GPU and CPU processing times shows that GPUs are faster because they can work with multiple images at the same time. GPUs are good at graphics tasks, which makes them better at quickly switching between images. As we process more images, the GPUs remain powerful and fast. The use of GPUs makes image processing faster and more responsive, showing the importance of dedicated hardware for better performance.

4 Possible shortcomings and improvements

1. Kernel Optimization: In my project the CUDA kernels use shared memory, but they have fixed block and thread configurations. If the block size gets optimized based on the image size, it can improve the performance and the usage of the GPU.

2. Image input support: My project only supports the input of JPEG images and lacks the support for other commonly and widely used image formats like RAW, PNG and TIFF. Applying this would be an improvement.

3. Providing a GUI: Providing an user input would increase the user friendliness and enhancing it in a pleasure usage of the application.